

INFO0027: Programming techniques, Algorithmics

Mohamed-Khalil Ankri and Robin Kulczycki

Academic year 2025-2026

Software Patterns: File Deduplication Service



Abstract

This report documents the design and implementation of a file deduplication service developed for the INFO0027 course. The service is built around a flexible, maintainable architecture that satisfies three stakeholders with diverging requirements: a Frontend team expecting a JSON-based API, a Storage team needing on-the-fly duplicate detection during file uploads, and Photography clients requiring perceptual image similarity grouping.

The system is structured around four design patterns which are Strategy, Factory, Adapter, and Service Provider, that keep the codebase modular and extensible. The production code spans roughly 600 lines across nine classes, passes all 17 automated functional tests on the Gradescope harness, and ships as a self-contained fat JAR with zero external runtime dependencies.

1 Architecture

This section describes the overall structure of the service, the role of each component, and how they interact at runtime. The design is organized around a clear separation of concerns, where each class has a single well-defined responsibility and communicates only with the layer directly below it.

1.1 Components

The system is organized around five concerns, each handled by a dedicated class or group of classes. Figure 1 shows the full structure.

- **Composition Root:** `DeduplicationBootstrapImpl` is the single entry point discovered by the test harness via `ServiceLoader`. It is responsible for building the entire object graph when `initialize(VirtualFileSystem)` is called, then exposing the two public-facing adapters. It holds no business logic of its own.
- **Adapters:** `FrontendGateImpl` translates incoming JSON strings into typed calls on the core service, and `StorageCheckerImpl` translates raw filesystem `Path` inputs into the same core. Each adapter is responsible only for its own protocol validation, including parsing, field checking, and error formatting.
- **Core Service:** `DeduplicationService` is the orchestrator. It uses `VfsWalker` to collect all regular files under a given path, then delegates grouping entirely to a `DeduplicationEngine`. It depends only on interfaces, never on concrete engines or adapters, keeping the two axes of change independent.
- **Engines:** `DeduplicationEngine` is the abstraction over “*given a list of files, return groups of duplicates*”. `ExactDeduplicationEngine` groups byte-identical files using SHA-256, with a size pre-filter that skips hashing for files with unique sizes. `SimilarDeduplicationEngine` fingerprints images using a 64-bit average hash (aHash) and clusters them by Hamming distance via union-find.
- **Utilities:** `VfsWalker` performs an iterative depth-first traversal of the `VirtualFileSystem`, and `Json` provides a minimal hand-rolled JSON parser and serializer with zero external dependencies. Centralizing traversal means engines never re-implement walking, and centralizing JSON handling means the project ships as a thin JAR with no shading of third-party libraries required.

1.2 UML Class Diagram

Figure 1 provides a visual overview of the system structure. The yellow cluster groups the interfaces and records provided by the course, which our code implements or depends on. The remaining classes are entirely our own. Dashed open-headed arrows represent realization (`implements`) and solid open-headed arrows represent dependencies between classes.

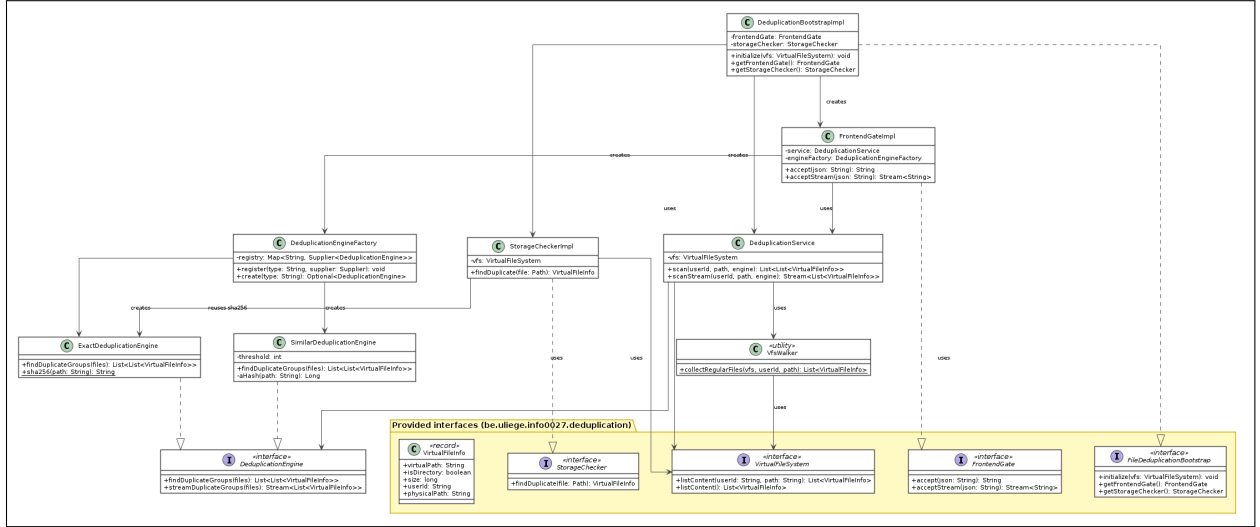


Figure 1: UML class diagram of the File Deduplication Service.

1.3 Request Lifecycle

This subsection describes how a typical request flows through the system at runtime, from the initial JSON input to the final response.

Taking the following request as an example:

```
{"action": "scan_duplicates", "scan_type": "exact", "path": "/docs", "user": "alice"}
```

The flow proceeds as follows. `FrontendGateImpl#accept` parses the JSON and validates all required fields, returning a structured error response on any failure. It then asks `DeduplicationEngineFactory` for the engine matching `scan_type`; an unknown type produces an *Unsupported scan_type* error. The resolved engine is passed to `DeduplicationService#scan`, which calls `VfsWalker` to collect every regular file under `/docs` for `alice`, runs the engine on that list, and returns the grouped results. `FrontendGateImpl` then serializes the groups back to JSON and returns.

The streaming variant `acceptStream` follows the same path but returns a `Stream<String>` where each element is one duplicate group serialized as a JSON array, emitted lazily as groups are produced.

The Storage team entry point `findDuplicate(Path)` takes a different entry point but reuses the same SHA-256 routine from `ExactDeduplicationEngine`. It walks every user's tree via `VirtualFileSystem#listContent()`, applies a size pre-filter before hashing, and returns the first matching `VirtualFileInfo` found.

2 Design Patterns

This section describes the design patterns applied in our implementation. For each pattern, we explain its purpose in the context of the project, where it is applied, and the trade-offs we considered.

2.1 Strategy

The Strategy pattern is applied through the `DeduplicationEngine` interface. The same input, a list of `VirtualFileInfo` objects, admits multiple algorithms selected at runtime from the `scan_type` field of the request. `ExactDeduplicationEngine` and `SimilarDeduplicationEngine` are two concrete strategies that can be swapped without any change to the calling code. The brief explicitly requires swappable engines, and adding a new one in the future is purely additive: one new class implementing `DeduplicationEngine` and one registration line in the factory. The trade-off is that engines cannot share intermediate state across calls, which is acceptable since our scans are fully independent.

2.2 Factory (Registry Variant)

The Factory pattern is applied through `DeduplicationEngineFactory`, which decouples the frontend from concrete engine classes. Instead of a hardcoded `switch` statement, the factory maintains a `Map<String, Supplier<DeduplicationEngine>` registry. New engines are registered with a single line and a `scan_type` string, with no existing code needing modification. This satisfies the Open/Closed Principle: the system is open for extension but closed for modification. The trade-off is reduced compile-time checking of supported types, which is mitigated by the explicit *Unsupported scan_type* error path returned to the caller.

2.3 Adapter

The Adapter pattern appears in two places. `FrontendGateImpl` adapts the JSON-string protocol required by the Frontend team to our typed internal core, and `StorageCheckerImpl` adapts the raw `Path`-based contract required by the Storage team to the same core. Splitting these two adapters keeps the core service completely ignorant of both protocols. The trade-off is some duplication of input validation boilerplate, which we accept because each adapter validates a structurally different shape of input.

2.4 Service Provider (ServiceLoader)

The Service Provider pattern is applied through Java’s built-in `ServiceLoader` mechanism, which is required by the test harness. `DeduplicationBootstrapImpl` is declared as a service implementation in `META-INF/services/` under the fully-qualified interface name `be.uliege.info0027.deduplication.FileDeduplicationBootstrap`. This makes the harness completely independent of our internal class names and package structure, and keeps the production wiring as a single line of resource configuration. The trade-off is that misconfiguring the services file produces a silent failure at runtime rather than a compile-time error.

3 Quality Assurance and Test Plan

This section describes our quality assurance strategy for the project. We selected three quality characteristics from ISO/IEC 25010:2024 that best reflect the expectations of this project: functional suitability, maintainability, and reliability. For each characteristic, we describe the relevant sub-characteristics, how we track them, and how we ensure they are met.

3.1 Functional Suitability

The relevant sub-characteristics are functional completeness and functional correctness. Functional completeness ensures that every required action returns a well-formed response, and functional correctness ensures that the responses match the specification.

We track these with a layered test suite. Unit tests target each engine in isolation, covering edge cases such as empty input, all unique files, three-way duplicate groups, and files with identical size but different content. Integration tests exercise the full `FrontendGate` JSON contract, including both scan types, all documented error paths (*Invalid JSON*, *Missing action*, *Missing user*, *Unsupported scan_type*), path-scoped scans, and the streaming variant. A `ServiceLoader` smoke test verifies that the provider configuration is correct so the harness can discover the bootstrap. All tests run against the `InMemoryVfs` fixture backed by real files on a `@TempDir`, so the SHA-256 and `ImageIO` code paths are exercised end-to-end. The full suite of 11 local tests passes in under one second on our development machine, and all 17 automated tests on the Gradescope harness pass as well.

3.2 Maintainability

We track four sub-characteristics. *Modularity*: no class has more than two direct collaborators of its own, verified by inspection of the UML diagram. *Reusability*: `VfsWalker` and the `sha256` utility are shared by otherwise unrelated components, avoiding code duplication. *Modifiability*: only interfaces are exposed between layers, so a concrete class can be replaced without touching its callers. *Testability*:

`VirtualFileSystem` is injected into every component that needs it, making each component independently testable with a mock or in-memory VFS implementation.

3.3 Reliability

The relevant sub-characteristics are fault tolerance and recoverability. Both engines treat hash or image decode failures as a silent skip rather than aborting the entire scan. Inner helpers return `null` on `IOException` and the caller simply moves on to the next file. `StorageCheckerImpl` returns `null` on any IO error, which correctly maps to the “no duplicate found” semantics of the `StorageChecker` contract. We rely on JDK cryptographic and IO primitives for correctness and do not test those independently.

3.4 Test Tooling

Tests are run via `mvn test` on JDK 25 using Maven Surefire. The test logs produced by a representative run are included in Annex A.

4 Design Process and Challenges

This section describes the key decisions we made during development, the challenges we encountered, and how we resolved them.

4.1 Initial Design and Iterations

Our first version put deduplication logic directly inside `FrontendGateImpl`. This immediately exposed the problem that the three stakeholders create: the Storage team’s interface takes a `Path`, not a JSON string, and the Photography clients want a different equivalence relation. Both would have forced duplicated logic in different parts of the codebase. Extracting the `DeduplicationEngine` Strategy and the `DeduplicationService` orchestrator resolved both issues at once and gave us a clean separation between detection and delivery.

A second iteration concerned `VfsWalker`. Initially each engine called `vfs.listContent()` itself to traverse the file tree, but the moment we implemented the second engine we found ourselves duplicating the same recursive traversal logic. Extracting a single `VfsWalker` utility eliminated the duplication and turned the engines into pure list-to-groups functions, which also made them significantly easier to test in isolation.

4.2 Performance Decisions

A deliberate optimization in `ExactDeduplicationEngine` was to bucket files by size before hashing. The size field is free from the VFS metadata, and on realistic file trees the majority of files have unique sizes, meaning SHA-256 work drops to near zero for most files without any change to the external API.

4.3 Similarity Engine

OpenCV was the natural choice suggested by the brief for the similarity engine, but it adds a 50+ MB native dependency that complicates the fat JAR and introduces platform-specific packaging concerns. We instead implemented a 64-bit average hash (aHash) with Hamming-distance clustering in roughly 70 lines using only `javax.imageio` from the JDK. The algorithm resizes each image to 8x8 pixels, converts it to grayscale, and builds a 64-bit fingerprint by comparing each pixel to the mean brightness. Two images are considered similar if their fingerprints differ by at most 5 bits out of 64, which corresponds to roughly 92% similarity. The threshold is exposed as a constructor parameter for easy tuning without any architectural change.

4.4 JSON Handling

Rather than pulling in a third-party library such as Gson or Jackson, we wrote a minimal hand-rolled JSON parser and serializer in the `Json` utility class. This keeps the project free of external runtime dependencies and allows the fat JAR to remain thin. The trade-off is that we maintain our own parser, but the input format is small and well-defined enough that this was judged worthwhile.

5 Limitations and Known Issues

This section discusses the known limitations of our current implementation and areas that could be improved in a production context.

5.1 Streaming is Not Truly Lazy

Both `scan` and `scanStream` in `DeduplicationService` call `VfsWalker.collectRegularFiles()` which collects all files into a list before any grouping begins. This means `acceptStream` is not truly lazy — it loads the entire file list into memory before emitting the first group. For a production system with very large file trees this could be a memory concern. A genuine streaming implementation would require a different input type for the engines, such as `Stream<VirtualFileInfo>` instead of `List<VirtualFileInfo>`.

5.2 Similar Engine Uses Single-Link Clustering

The similarity engine uses single-link clustering via union-find, which means a chain of pairwise-close images can transitively form one large group even if the two endpoints differ noticeably. For most photo libraries this matches user expectations, since burst-mode photos all belong together, but a stricter consumer might prefer complete-link clustering or a tighter threshold. The threshold and the hash function are both exposed as constructor parameters in `SimilarDeduplicationEngine`, so this can be tuned without any architectural change.

5.3 StorageChecker Does Not Scale

`StorageCheckerImpl` walks every user's entire file tree on each call to `findDuplicate`. For a production system with millions of users and files this would be unacceptably slow. The correct approach would be to maintain a content-addressable index keyed on (size, SHA-256) updated incrementally on each upload. The current implementation is correct and sufficiently fast for the test sizes implied by the brief, and the `StorageChecker` interface remains unchanged if the implementation is replaced.

5.4 C Library Interoperability Not Implemented

The optional C library engine is not included. The `Strategy` interface already accommodates it — adding it requires one new class implementing `DeduplicationEngine` and one registration line in `DeduplicationEngineFactory`. The Java Panama bindings, the `.so` packaging, and the platform detection were judged outside the scope of the minimum required implementation.

5.5 Duplicate Escape Logic

The `Json` utility class contains two separate methods that implement the same character escaping logic: `quote()` inside the `Writer` class and the standalone `escape()` method. These should ideally share a single implementation to avoid the risk of one being updated without the other. This is a known maintainability issue that would be addressed in a future iteration.

Declaration on Generative AI

During the preparation of this work, the authors used Anthropic's Claude (Sonnet 4.5) to assist with code drafting and refactoring, generate JUnit test cases, produce a first draft of this report, and generate the

UML class diagram. After using this tool, the authors reviewed and edited all content as needed and take full responsibility for the publication's content.

A Test Logs

The following output was produced by running `mvn test` on our development machine with JDK 25. All 11 local tests pass in under one second.

```
[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running be.uliege.info0027.dedup.FrontendGateImplTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.268 s
[INFO] Running be.uliege.info0027.dedup.StorageCheckerImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0,
      Time elapsed: 0.017 s
[INFO]
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

B Build Instructions

To build and test the project, run the following commands from the project root:

```
mvn clean package    # produces Deduplicator.jar at the project root
mvn test             # runs the full JUnit test suite
```

The `ServiceLoader` provider configuration file is located at:

```
src/main/resources/META-INF/services/
be.uliege.info0027.deduplication.FileDeduplicationBootstrap
```

containing the single line:

```
be.uliege.info0027.dedup.DeduplicationBootstrapImpl
```

which is the class instantiated by the test harness via `ServiceLoader`.